To help convergence, we first allow each agent to select an action based on these random mean fields, and we store the action in the temporary grid copy, `grid__`, so that the main grid doesn't change until all agents have made a final decision about which action to take. After each agent has made a tentative action in `grid__`, we update the second temporary grid copy, `grid_` which is what we're using to calculate the mean fields. In the next iteration, the mean fields will change, and we allow the agents to update their tentative actions. We do this a few times (controlled by the `num_iter` parameter) to allow the actions to stabilize around a near optimal value based on the current version of the Q function. Then we update the main grid and collect all the actions, rewards, mean fields, and `q_next` values ($V(S_{t+1})$ and add them to the experience replay buffer.

Once the replay buffer has more experiences than the batch size parameter, we can begin training on mini-batches of experiences. We generate a list of random index values and use these to subset some random experiences in the replay buffer. Then we run one step of gradient descent as usual. Let's run the training loop and see what we get.

```
>>> fig,ax = plt.subplots(2,1)
>>> ax[0].plot(np.array(losses).mean(axis=0))
>>> ax[1].imshow(grid)
```

It worked! You can see from figure 9.22 that all but three of the electrons (agents) have their spins aligned in the same direction, which minimizes the energy of the system (and maximizes the reward). The loss plot looks chaotic partly because we're using a single DQN to model each agent, so the DQN is sort of in a battle against itself when one agent is trying to align to its neighbor but its neighbor is trying to align to another agent. Some instability can happen.

Figure 9.22. The top graph is the loss plot for the DQN. The loss doesn't look like it is converging, but we can see that it does indeed learn to minimize the energy of the system (maximize reward) as a whole in the bottom panel.